

Character Input and Output in Files

ANSI C provides a set of functions for reading and writing character by character or one byte at a time. These functions are defined in the standard library.



Key Functions



getchar()



putchar()



printf()



Reading single characters from standard input



scanf()

Reading formatted input from standard input

Writing single characters to standard output

Writing formatted output to standard output



1/5



The getchar() Function

This will get a character from the keyboard, waiting as long as required for a key to be pressed. When the character is found, its corresponding bit pattern (usually ASCII value) is assigned to a

variable.



Syntax

```
int getchar();
```

getc() is used to read a character from a file and putc() is used to write a character to a file. Their syntax is:

```
int putc(int ch, FILE *stream);  
int getc(FILE *stream);
```

If the function encounters an error, such as an end of file marker, it returns a value EOF (this type of error is unusual from a keyboard!). EOF is a constant defined in stdio.h header file described in section C.

It is possible to use `getchar()` without assigning the resulting value to any variable, such as in the statement:

```
getchar();
```

2/5

The `putchar()` Function

`putchar` outputs a character to the screen as follows:

```
putchar(character_expression);
```

This will output to the screen whatever character has a bit pattern that corresponds to the value of the specified parameter in ().



Examples

```
int fred = 65;
putchar('x'); /* outputs character 'x' */
putchar(fred); /* outputs character 'A' */
putchar(2); /* will not put a 2 onto the screen */
```

1. putchar('x') is not the same as putchar(x).

2. 'x' is not the same as "x".

3. putchar(2) will not put a 2 onto the screen.

The bit pattern of the number 2 does not correspond to the ASCII value of a character that is



"printable". i.e. Nothing will appear on the screen.

3/5

The printf() Function

printf outputs a string of characters to the screen.

A string is a collection of one or more characters with a hidden zero byte at the end.

A string with one character in it is not the same as a single character. Double quotes, " " enclose a string, single quotes ' ' enclose a character.

The `\n` in the above example makes sure the output is on a new line. Otherwise, it would have



continued where it previously left off.

If the string contains a %d then the output is modified by substituting the value of the next parameter in place of %d. For each further %d another parameter value is substituted.



Example

```
printf("Add %d and %d to get %d.",a,b,a+b);
```

An expression can be used for a printf parameter as in a+b.



The scanf() Function

The `int scanf(const char *format, ...)` function reads input from the standard input stream `stdin` and scans that input according to the format provided.

The format can be a simple constant string, but you can specify `%s`, `%d`, `%c`, `%f`, etc., to print or read strings, integers, characters, or floats, respectively. There are many other formatting options available which can be used based on requirements.



Example

```
#include <stdio.h>

int main( )
{
    char str[100];
    int i;
    printf( "Enter a value :");
    scanf("%s %d", str, &i);
    printf( "\nYou entered: %s %d ", str, i);
}
```



```
return 0;  
}
```

